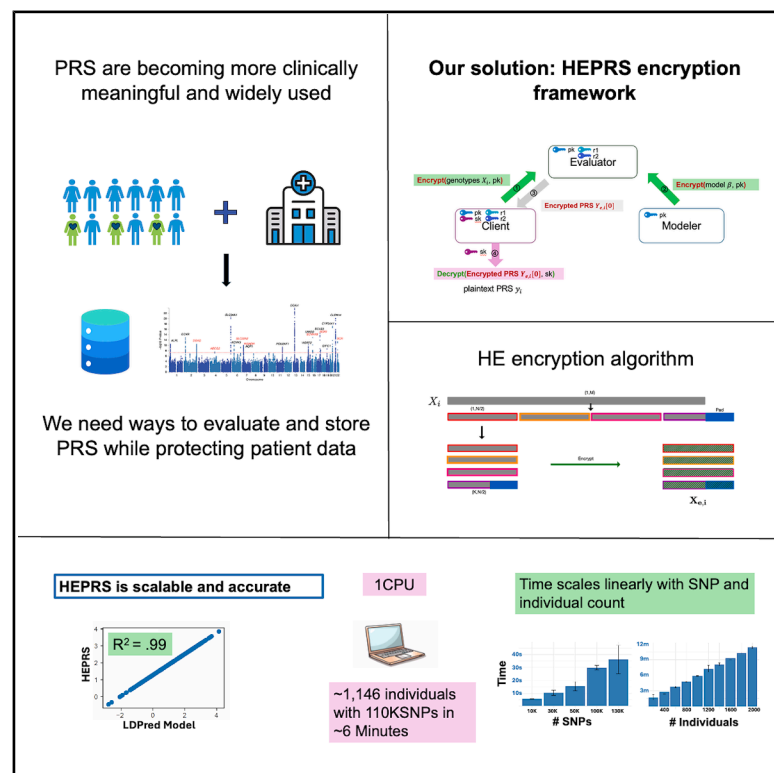


Homomorphic encryption enables privacy preserving polygenic risk scores

Graphical abstract



Authors

Elizabeth Knight, Jiaqi Li, Matthew Jensen, Israel Yolou, Can Kockan, Mark Gerstein

Correspondence

mark@gersteinlab.org

In brief

Knight et al. present HEPRS, a fully homomorphic encryption pipeline that computes polygenic risk scores directly on encrypted genotypes. Using a 110k SNP schizophrenia model, encrypted and plaintext PRSs closely match while preserving privacy, enabling practical and secure PRS inference for clinical genomics.

Highlights

- FHE computes PRS on encrypted genomes with near-zero accuracy loss
- HEPRS runs a 110k-SNP schizophrenia PRS in minutes on a single CPU
- Encrypted and plaintext PRSs correlate at $r > 0.999$ with tiny MSE
- Runtime and memory scale linearly with SNP count and cohort size



Article

Homomorphic encryption enables privacy preserving polygenic risk scores

Elizabeth Knight,¹ Jiaqi Li,¹ Matthew Jensen,¹ Israel Yolou,¹ Can Kockan,² and Mark Gerstein^{1,3,4,5,6,7,*}

¹Program in Computational Biology and Biomedical Informatics, Yale University, New Haven, CT 06520, USA

²Broad Institute of MIT and Harvard, Cambridge, MA 02142, USA

³Department of Molecular Biophysics and Biochemistry, Yale University, New Haven, CT 06511, USA

⁴Department of Computer Science, Yale University, New Haven, CT 06511, USA

⁵Department of Biomedical Informatics and Data Science, Yale University, New Haven, CT 06511, USA

⁶Department of Statistics and Data Science, Yale University, New Haven, CT 06511, USA

⁷Lead contact

*Correspondence: mark@gersteinlab.org

<https://doi.org/10.1016/j.crmeth.2025.101271>

MOTIVATION Clinically useful polygenic risk scores increasingly depend on the use of large, distributed, and sensitive genomic datasets and collections of genomes. Fully homomorphic encryption allows computing on encrypted data so that neither genomes nor models need to be revealed. We demonstrate that, even at the scale required for clinically relevant polygenic risk score calculations, our framework enables secure computation with negligible accuracy loss and feasible computational resource demands.

SUMMARY

Polygenic risk score models (PRSs) are important tools in precision medicine, enabling personalized risk prediction; however, they raise privacy concerns. Fully homomorphic encryption (FHE) provides a potential solution, allowing computation on encrypted genomic data. Here, we develop an open-source implementation of FHE for PRS (HEPRS), available online. HEPRS involves a three party system: clients (clinicians handling sensitive genetic data), modelers developing a PRS (academics), and evaluators (a local hospital running the models while maintaining confidentiality). We apply HEPRS to synthetic datasets and a 110,000 single-nucleotide-polymorphism (SNP) model for schizophrenia and show that encrypted and plaintext PRSs agree closely. We investigate encryption parameters that influence computational accuracy, memory, and time, demonstrating that HEPRS is practical to use on a single CPU. These results show that FHE enables realistic, privacy-preserving PRSs with negligible accuracy loss, supporting secure and scalable genomic analytics.

INTRODUCTION

Advances in genetic sequencing technologies have significantly reduced both time and cost, making sequencing a commonly used practice in biomedical research and precision medicine.¹ Genomic sequencing data have the potential to provide valuable insights into biological processes and clinical treatment, especially with the multitude of associations established between genomic loci and traits or diseases through genome-wide association studies (GWASs).² Such studies hold promise for various clinical applications, from more accurate molecular diagnosis to drug target prioritization.^{3,4} A meaningful clinical advance stemming from GWASs is the development of polygenic risk scores (PRSs), which are statistical models that utilize genetic variants to predict an individual's risk of having a particular trait or condition, such as disease, height, or intelligence.⁵ PRSs have the potential to bring personalized approaches to health

and wellness, from early detection and prevention through lifestyle changes or medical interventions to more effective and individualized care. PRSs can also identify subgroups of individuals more likely to respond to certain medications or treatments, improving patient outcomes.⁶ For example, a recent study found that PRSs can indicate future heart attack risk, prioritizing patients for preventive statin treatments.⁷ Similarly, PRSs for breast cancer can identify subgroups of individuals with high predicted incidence for early screening interventions,⁸ further highlighting the utility of PRSs in a clinical setting.

GWASs for most common traits (such as height) and diseases (such as breast cancer) typically require genomic data from tens of thousands of individuals at hundreds of thousands of loci for accurate results.^{9,10} The sheer amount and complexity of these genetic data can strain local computational resources and often require the involvement of third parties or cloud-based infrastructures, thereby increasing the risk of genetic privacy violations.¹¹



For example, the UK Biobank contains more than 400,000 samples, whose data are often too large to be fully downloaded, requiring the use of the DNANexus platform for downstream analysis.¹² Protection of patient genomic data is particularly important for PRS calculation, where an individual's genome is compared against summary results derived from a large, often external cohort. In the face of quantum computing advancements, quantum-secure encryption has become important in ensuring the privacy of data transfers across systems.¹³ As genomic data become increasingly important in a healthcare setting, hospitals must adopt advanced encryption solutions to ensure the privacy and security of patient data, especially genomic data, while also reducing costs and improving the efficiency of clinical research. Thus, advanced encryption methods are needed to protect sensitive genomic data from unauthorized access and misuse while allowing physicians to calculate and access their patients' PRSs securely and confidentially. With the increasing use of PRS models trained on confidential patient information, establishing model security is also critical to protect clinical data and prevent potential model inversion attacks.¹⁴

Despite the potential of PRSs in clinical applications, ensuring the privacy of genomic data remains a challenge. Various methods have been proposed to address genomic privacy concerns, including cryptographic techniques such as homomorphic encryption. Previous applications of fully homomorphic encryption (FHE) in healthcare have set the stage for advancing patient data privacy. For example, Kara et al.¹⁵ initially leveraged this technology to securely handle vital sign data from medical devices. Later, Raisaro et al.¹⁶ extended the use of FHE to protect genetic data within hospital systems, enabling researchers to access GWAS summary statistics without compromising patient privacy. Naveed et al.¹⁷ provided a comprehensive survey of privacy-preserving methods in genomics, highlighting cryptographic approaches to enhance data security. Similarly, Bonomi et al.¹⁸ discussed the privacy challenges in genomic data sharing and suggested homomorphic encryption as a promising solution for secure computations on genomic data, although they did not implement it in their study. Previous work has also applied homomorphic encryption to genomic data analysis tasks other than PRS calculation. Kim and Lauter¹⁹ utilized the Brakerski-Gentry-Vaikuntanathan (BGV) and Yet Another Somewhat Homomorphic Encryption (YASHE) homomorphic encryption schemes to securely compute minor allele frequencies and chi-square statistics in GWAS, as well as to calculate Hamming and approximate edit distances between DNA sequences. Their experiments with datasets containing up to 5,000 DNA sequences demonstrated the feasibility of using homomorphic encryption in genomic analysis, although PRS calculations were not performed. In clinical settings, McLaren et al.²⁰ used homomorphic encryption to securely evaluate specific genetic loci associated with drug resistance or treatment response in HIV patients. This approach pioneered privacy-preserving genomic testing but did not extend to PRSs. Blatt et al.²¹ built a privacy-preserving pipeline for large-scale GWAS using an optimized variant of the Cheon-Kim-Kim-Song (CKKS) homomorphic encryption scheme. The scheme allows researchers to obtain GWAS summary statistics without ever seeing an in-

dividual's genomic data, as the encrypted genomic data reside only in the cloud. The authors noted that PRSs can be computed once the odds ratios are decrypted, but a fully end-to-end encrypted pipeline for PRS was left for future work. Implementing FHE in PRS calculation could significantly enhance data security by maintaining patient genomes in encrypted formats, potentially reducing vulnerabilities and costs when facilitating the secure transfer and manipulation of encrypted genomic data.

Homomorphic encryption is a promising solution to the problem of preserving confidentiality while enabling important computations on sensitive genomic data. This type of encryption allows third parties to perform computational functions on encrypted data (ciphertext) without first decrypting it. It addresses privacy concerns in digital communication by preserving confidentiality during data manipulation without exposing it to unintended parties.²² There are several types of homomorphic encryption, including partially homomorphic, somewhat homomorphic, leveled fully homomorphic, and FHE, each offering varying degrees of computation on encrypted data.²² Many commonly used homomorphic encryption schemes rely on the Ring Learning With Errors (RLWE) problem, which deliberately introduces a small, controlled amount of noise into ciphertexts to guarantee security as each computational operation is performed (with addition contributing less noise than multiplication). The noise accumulates, and the RLWE framework ensures that this noise cannot be reversed or "subtracted out" without the secret key, thereby preventing backward reconstruction of the original plaintext. However, different types of homomorphic encryptions have limitations regarding the set of circuits, or the range and complexity of computational sequences that can be performed on encrypted data, and the types of gates, or basic operations (such as addition or multiplication), that are supported in these computations.^{23,24}

Here, we apply FHE specifically to genotype data for secure PRS calculation and phenotype prediction. Utilizing the CKKS protocol for FHE within the Lattigo library,²⁵ we implement an approach that allows for the computation of PRSs directly on encrypted genetic datasets, obtaining results securely and privately. Our approach entails the propagation of encrypted data across three parties (clients with sensitive genetic data, modelers with existing PRS models, and evaluators to interpret PRS findings), preserving both model and genomic privacy when communicating PRS results back to patients.²⁶ While other studies have proposed homomorphic encryption of PRS models and provided proof-of-concept on limited artificial datasets,²⁷ we take advantage of the scalability of FHE to preserve the privacy of robust PRS models that contain a larger number of significant single-nucleotide polymorphisms (SNPs). In addition to robust assessment with synthetic datasets to show how our model scales with increased SNP numbers, we apply our FHE-based protocol to a 110k-SNP PRS model for schizophrenia,²⁸ which demonstrates accuracy in predicting schizophrenia risk in a cohort of over 1,200 individuals, with minimal decrease in performance compared to non-encrypted PRS. We also thoroughly investigate the trade-offs associated with encryption parameters that influence computational accuracy, memory, and time for large PRS models, which are relevant for real-world

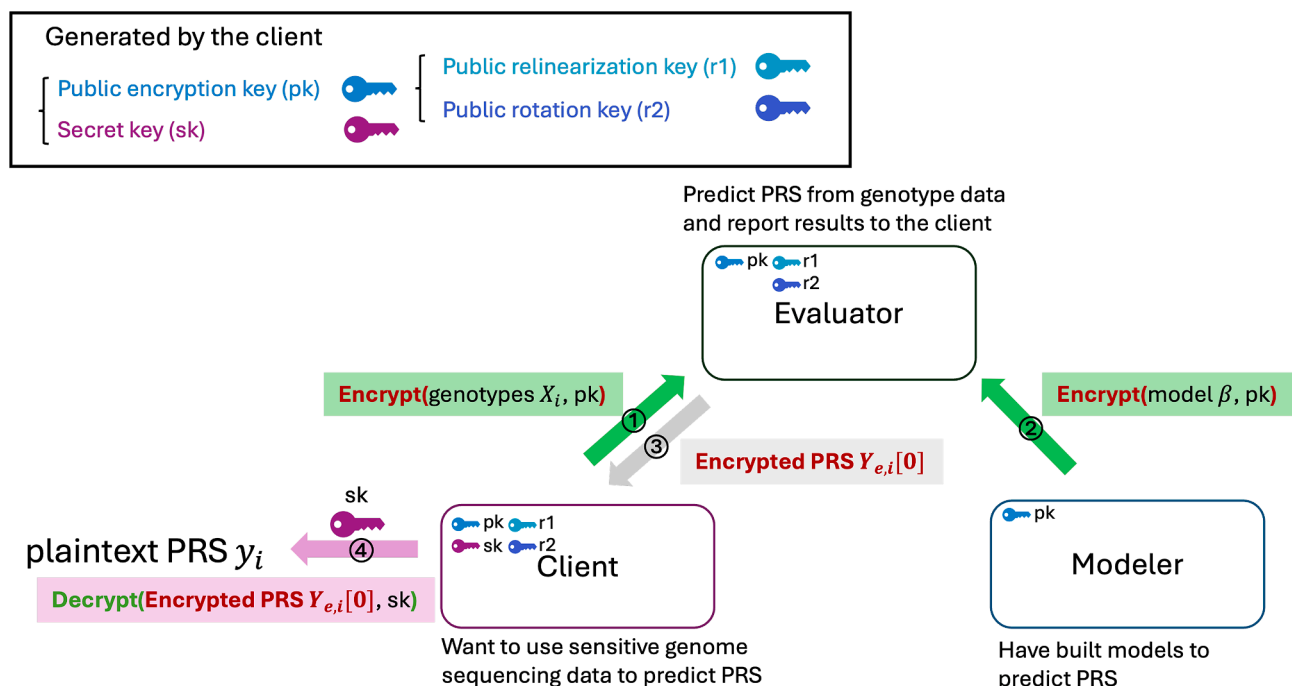


Figure 1. Schematic illustration of the encryption setting

The public encryption key (pk), public relinearization key (r1), public rotation key (r2), and secret key (sk) are generated by the client. The public keys (pk, r1, and r2) are shared with all three parties, while the secret key is only known to the client. The client performs homomorphic encryption of their sensitive genetic data using the public key, and the modeler encrypts their model using the same public key. The clients' genomic data and the model are sent to the evaluator in an encrypted form. The evaluator cannot decrypt the data or the model without the secret key. However, the evaluator is able to evaluate the genetic data and obtain model predictions, which remain encrypted. The encrypted phenotype predictions are reported back to the client, who can decrypt the data using the secret key and read the predictions.

healthcare applications. Our FHE protocol can preserve PRS while operating within a reasonable time frame; in fact, evaluating the PRS of one individual with the 110k-SNP model required only 4.9 s and 3.3 GB of memory with optimal parameters, while calculations for 1,000 genotypes took less than 5 min and used 130 GB of memory on a conventional laptop (MacBook Pro 2021). By using FHE to obtain meaningful results from realistic PRS models while maintaining the security of the underlying genomic datasets, our study provides valuable insights into the potential applications of FHE in genomics and healthcare. An implementation of our FHE protocol is available as an open-source software package called HEPRS at <https://github.com/gersteinlab/HEPRS>.

RESULTS

Overview

Our work establishes a privacy-preserving protocol utilizing homomorphic encryption to predict phenotype risk from genomic data by involving three independent parties: the client, the modeler, and the evaluator (Figure 1). Our protocol delineates distinct roles—clinicians could be clients providing genomic data, researchers or private companies could serve as modelers creating the PRS, and hospitals or centralized systems could act as evaluators processing the encrypted genomes. This

approach ensures effective use of genomic information while prioritizing patient confidentiality.

This protocol utilizes the CKKS encryption scheme,²⁹ which is quantum secure and allows for a broader range of bootstrapped computations on encrypted data than other homomorphic encryption methods. Using both synthetic and real genomic datasets, we showed that our encryption scheme efficiently calculates PRS values without sacrificing privacy or accuracy compared with traditional non-encrypted PRS methods. We further systematically analyzed the various encryption parameters used in our method to report their effects on accuracy (R-squared and area under the receiver operating characteristic curve [AUROC], memory usage, and computational time). Overall, our findings underscore the practicality of employing FHE in clinical genomics, particularly for PRS calculations where data privacy is paramount, without compromising computational integrity.

We evaluated the performance of our FHE-based PRS calculation method using both synthetic datasets and a real dataset for schizophrenia risk. We first applied our method to a synthetic dataset created using HAPGEN2,³⁰ which contained 2,000 individuals and a range of SNPs for three artificial phenotypes (referred to as phenotypes 0, 1, and 2). We then applied our method to a dataset of over 1,100 individuals from the PsychENCODE Consortium,³¹ evaluated against a 110k-SNP PRS model for schizophrenia.²⁸ We compared the

performance of our encrypted PRS with the non-encrypted PRS in terms of score correlation (Pearson r) and mean square error (MSE) in phenotypic variance explained by the scores.

Encrypted PRS computation overview

The FHE encryption framework involves the client, the modeler, and the evaluator²⁶ (Figure 1). The roles within our protocol can be adapted to various configurations, aligning with the operational structures of clinical genomics environments. For instance, the client could be a clinician tasked with diagnosing or treating a patient, ensuring that the genomic data are utilized effectively while maintaining patient confidentiality. The modeler could represent a centralized data repository, akin to the UKBiobank resource, which manages PRS calculations and the underlying summary statistics for various disorders. The evaluator could be a hospital's biobank, which can securely access and process a patient's encrypted genome in conjunction with their electronic health record. In this case, the client has genomic data and wants phenotype predictions, the modeler trains a model for predictions, and the evaluator evaluates the model using the client's data and reports the forecast to the client.

Privacy is protected throughout the framework using homomorphic encryption, where the client's data and model are encrypted, and computations (i.e., calculating the PRS) are performed only in an encrypted form (Figure 1). Specifically, the client first generates a public key and a secret key and encrypts the data using the public key. The client then shares the public key with the modeler and the evaluator, and the modeler encrypts their model using the same public key. The client and modeler both share their encrypted data with the evaluator. The evaluator performs the computation using the encrypted model, encrypted data, and public key. Finally, the evaluator returns the encrypted predictions to the client, who then decrypts the results with their secret key. We assume no crosstalk between the client and the evaluator. It is also pertinent to acknowledge that the modeler, in handling GWAS summary statistics, is a custodian of sensitive information, as minor allele frequencies could allow for identifying individuals in the study.³² In this context, the modeler must trust that the evaluator will not share the encrypted model with the client. In this way, all data are encrypted and only the client can access the plaintext predictions (Figure 1).

Implementation of homomorphic encryption

To implement FHE, we first load and encrypt the genotype data on the client side. During this step, the client generates the secret decryption key (s_k), which is then used to derive the public encryption key (p_k), the public relinearization key (r_1), and the public rotation key (r_2). These public keys are shared with all parties. The client then encrypts the genotype data (X_i) to transform each individual's genotype into a two-dimensional ciphertext matrix ($X_{e,i}$). The chosen parameters determine the dimensions of the vector.

Next, the modeler loads and encrypts the PRS model β using the public encryption key (p_k) provided by the client. A similar scheme, dependent on the parameters, saves each model as a two-dimensional ciphertext matrix β_e .

After this, the evaluator loads and evaluates the encrypted model β_e on the encrypted genomes $X_{e,i}$. In this arrangement,

we assume that the evaluator and client are both trustworthy and cannot collude; the encrypted model is never shared with the client. The evaluator uses the public relinearization key r_1 and rotation key r_2 provided by the client, along with the encrypted model β_e and genome $X_{e,i}$, to perform a polynomial evaluation and obtain the encrypted phenotype (i.e., PRS value $Y_{e,i}[0]$). This step involves an inner product calculation performed using homomorphic encryption, as detailed below.

Finally, once the client receives the encrypted phenotype result $Y_{e,i}[0]$ (risk score for disease), the client decrypts the result with the secret key s_k , producing the final PRS y_i .

We implemented the protocol using Golang. Golang's suitability for cryptography is underscored by its comprehensive standard library, which includes support for common cryptographic algorithms, along with features such as concurrency support, inherent memory safety, and strong typing with compile-time checks.

Implementation of inner product using homomorphic encryption

We first encode both the model β and X_i genome for J individuals using the CKKS plaintext encoding scheme. The CKKS encryption parameter set used for our experiments is summarized in Table 2. During encoding, the integer-based plaintext vector X_i (representing genotype values of 0, 1, or 2) is encoded into the domain $\frac{Z_Q[X]}{X^N+1}$. This requires us to represent our vectors as polynomials where all coefficients are $\in Z_Q$ and all operations are performed as mod X^N+1 .

In the following section, we focus on the calculation of a single individual's PRS. Here, the encrypted genomic data matrix $X_{e,i}$ encapsulates the genotypes, while the model matrix β_e holds the beta values. Both matrices have dimensions $K \times N/2$, aligning with the encryption parameters outlined in Figure 2.

Algorithm 1 is instrumental in the PRS calculation by implementing the dot product of encrypted vectors. Within the for-loop, each row vector from the genomic data matrix $X_{e,i}$ is element-wise multiplied with the corresponding row from the model matrix β_e using the `MulRelinNew()` function from Lattigo.²⁵ This multiplication yields new vectors, $v_h = X_{e,i}[h] * \beta_e[h]$, representing the weighted contribution of each genotype to the PRS (here $*$ represents an element-wise multiplication). The resulting vector aggregates these products, accumulating the combined effect of genotypes and beta values on the PRS. The `Lattigo InnerSumLog()` function subsequently sums the polynomial terms within the result vector, reflecting the final summation of individual genetic risk factors. The `InnerSumLog()` operation is called once at the end, outside of the loop with $\log_2(N/2)$ rotations. This process effectively translates the encrypted genomic information and model predictions into a single PRS value for each individual. This function fails to perform some specialized processes currently executed by more advanced calculation methods such as the `plink2 clump` function³³; however, this process calculates a sufficient PRS that differs from the `Plink 2` model by only 0.000129%.

Overview of PRS derivations

We used two complementary approaches to assess the performance of our homomorphic encryption-based PRS calculations. We first generated synthetic datasets using HAPGEN2,³⁰

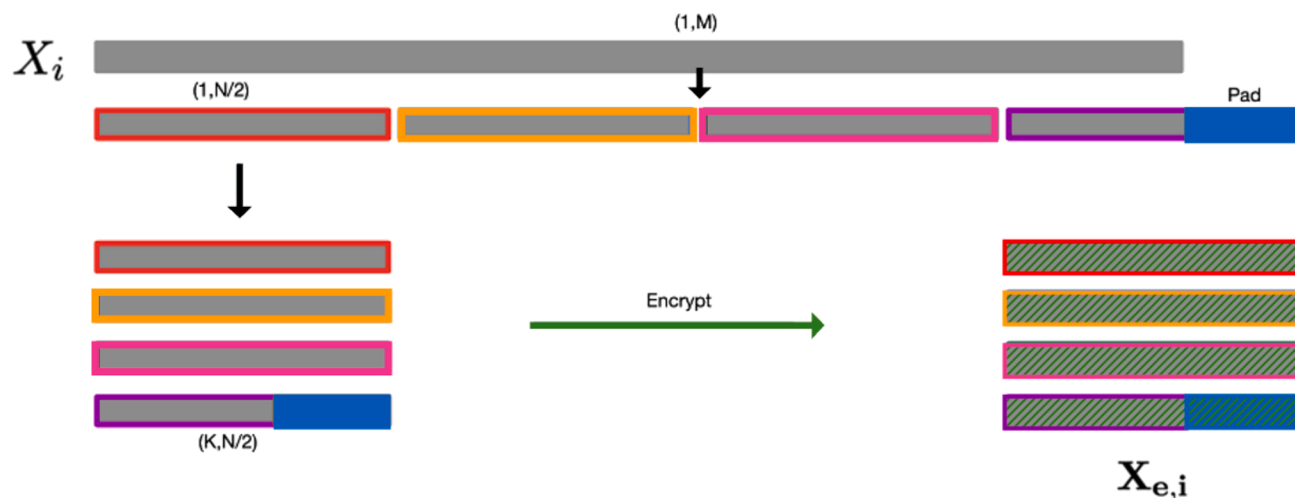


Figure 2. Encoding of genomic vector into encrypted matrix

This schematic shows how we encode the genomic vector X_i into a genomic matrix in the encrypted domain $X_{e,i}$. Green hash lines denote encrypted. Blue denotes the padding.

a simulation method that produces genotypes with realistic LD patterns, to further evaluate our approach under various conditions. We simulated datasets with varying numbers of SNPs (10,000–130,000) and sample sizes (200–2,000 individuals) and calculated PRSs for three artificial phenotypes with different levels of heritability using ridge regression. By assessing our FHE method on both real and synthetic data, we demonstrated its versatility and potential for secure and private PRS calculations in diverse scenarios. We also applied our method to a real-world dataset for schizophrenia, utilizing PRSs derived from a recent GWAS of the Psychiatric Genomics Consortium (PGC) of more than 320,000 individuals.²⁸ Specifically, we calculated PRS for 1,146 individuals (493 cases and 653 controls) from the PsychENCODE Consortium,³¹ using a standardized data processing pipeline based on best practice guidelines,³⁴ followed by LDpred2 for PRS calculation.³⁵ The resulting PRS model contained 110,258 SNPs and achieved a pseudo- R^2 value of 0.2229 in predicting schizophrenia diagnosis.

For both approaches, we found that our FHE procedure introduced only a trivial amount of error compared to the inherent uncertainty of the PRS value as determined from non-encrypted

approaches, despite the framework's computational and memory limitations with increasing data size. Specifically, the variability of the phenotype explained by PRS (R^2) from our homomorphic encryption framework closely matched that of the underlying PRS models for both synthetic and real data, indicating that the explained variance in the phenotype within the population remained consistent after encryption. For example, we observed a small MSE ($<1.5 \times 10^{-8}$) when applying FHE with varying R^2 values to the synthetic datasets (Table 1). After ridge regression analysis on the encrypted data, the R^2 values obtained for the three artificial phenotypes were 0.189, 0.466, and 0.822, respectively (compared with the preset values of 0.3, 0.6, and 0.9). Similarly, using the 110k-SNP dataset for schizophrenia PRS, our encrypted PRS values had an R^2 value of 0.2232 (with PRS, sex, age, and genotype principal components (PCs) as covariates), compared with 0.2226 for the non-encrypted model (Table S2). This corresponded to an MSE of 2.27×10^{-6} between the non-encrypted PRS and the encrypted PRS (for ring dimension 2^{13}). This minimal MSE demonstrated that the encrypted PRS data yielded comparable R^2 values to those obtained without encryption, suggesting that our FHE

Algorithm 1. Multiply vectors and add together

Input: $X_{e,i}$: fixed-length matrix ckks.Ciphertext β_e : fixed-length matrix ckks.Ciphertext K : integer

- 1: result = vector(length = $N/2$) ▷ this initializes a new ciphertext vector
- 2: for h in 1: K do
- 3: v_h = multiply $X_{e,i}[h]$ with $\beta_e[h]$ elementwise ▷ $\text{len}(v_h) = \text{len}(X_{e,i}[h])$
- 4: $Y_{e,i} = Y_{e,i} + v_h$ ▷ addition step
- 5: end for
- 6: $Y_{e,i}$ = polynomial rotational addition of ($Y_{e,i}$) ▷ InnerSumLog() with $\log_2(N/2)$ rotations
- 7: return ($Y_{e,i}[0]$) ▷ return the first element of the result vector (complex scalar)

Table 1. Performance of encrypted PRS models with ring dimension 2^{13}

Phenotype	Ring dimension	R^2 of plaintext model	R^2 of encrypted model	MSE between plaintext and encrypted model
0	2^{13}	0.18892353	0.18892715	1.397e-08
1	2^{13}	0.46645884	0.4664542	8.085e-09
2	2^{13}	0.8226163	0.8226161	1.168e-08

Performance of encrypted PRS models with ring dimension 2^{13} across three phenotypes with varying heritability ($h^2 = 0.3, 0.6, 0.9$). The R^2 of the encrypted model and MSE relative to the corresponding plaintext model are shown.

method introduces only a small error that would not lead to changes in the derived PRS. Figure 3A demonstrates that the LDpred2 and encrypted PRS methods both achieved an AUROC of 0.61. Additionally, Figure 3B shows a Pearson correlation of $r > 0.999$ between the encrypted and non-encrypted schizophrenia PRS. In terms of computational cost, Figure 3C

shows that the majority of time is spent on input encryption and encrypted computation, while model encryption and result decryption require comparatively little time.

Effects of input data size on runtime and memory for encrypted PRS

While several parameters may affect runtime and memory usage, our method was able to quickly and accurately calculate PRSs using encrypted genotypes and PRS models, even when implemented on a personal laptop. For instance, using a MacBook Pro (M1, 2020), our method required about 10 s and 3 GB of memory to calculate PRSs for one individual sample using the 110k schizophrenia PRS model (ring dimension 2^{13}). Calculating PRSs for a larger cohort of 1,146 samples on a single CPU (Intel 6234) required 6 min and about 65 GB of memory when using the 110k-SNP model with the ring dimension 2^{13} (Table S2). The run time for 2,000 samples in the 10k-SNP synthetic dataset took about 2 min for any of the three synthetic phenotypes.

Table S2 demonstrates that the protocol utilizing homomorphic encryption for dot product calculation was roughly three

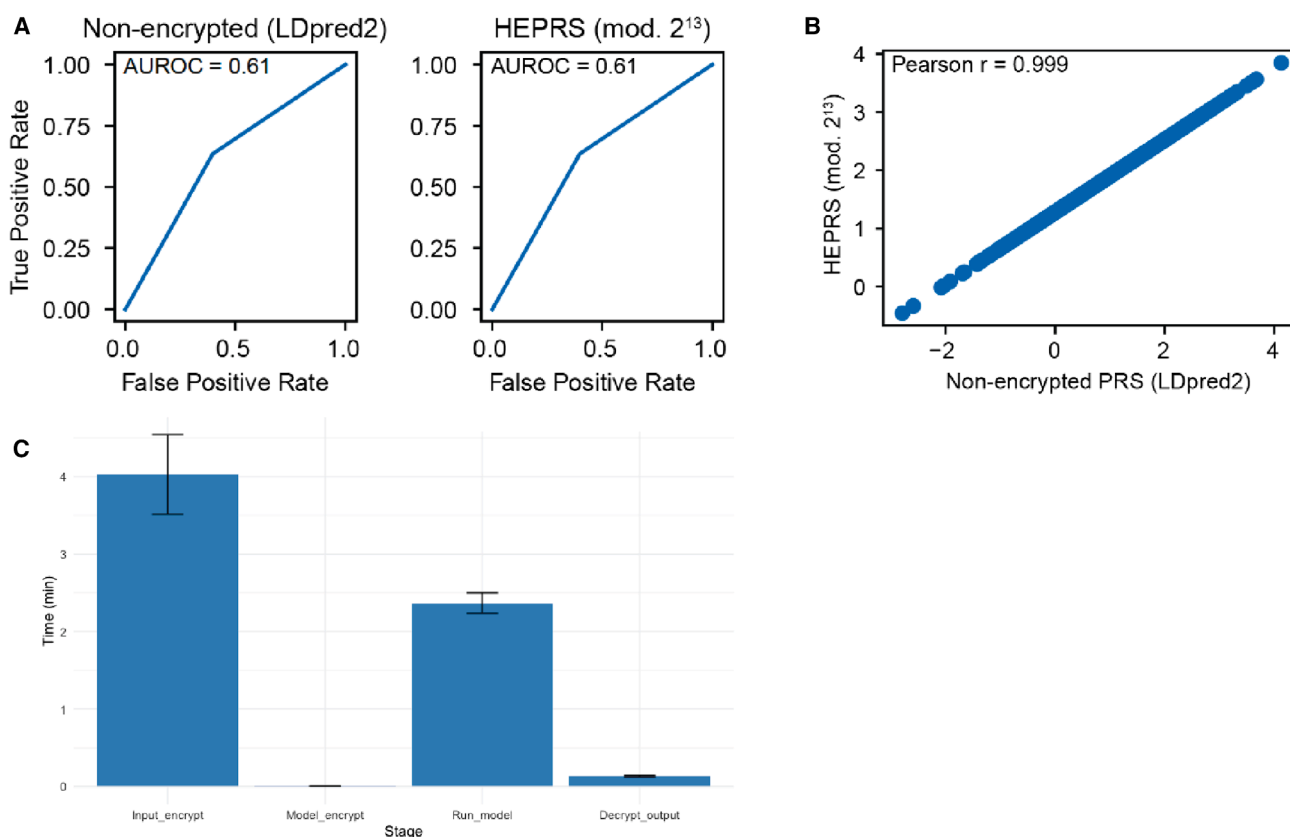


Figure 3. Accuracy and cost on the schizophrenia cohort

(A) Receiver operating characteristic curves for prediction of schizophrenia phenotypes based on non-encrypted schizophrenia PRSs (calculated with LDpred2) and PRSs calculated from HEPRS using the ring dimension parameter 2^{13} . PRS accuracy was assessed by considering individuals with >0.5 percentile scores for each calculation to have schizophrenia and comparing with the actual phenotypes. AUROC values were equal across all methods.

(B) Pearson correlations between encrypted schizophrenia PRSs with the ring dimension parameter 2^{13} and non-encrypted PRSs calculated with LDpred2 each show a high correlation ($r > 0.999$).

(C) Breakdown of computation time into different steps of the homomorphic encryption process using the ring dimension parameter 2^{13} . The time was calculated by assessing PRSs of 1,146 individuals using the 110k-SNP schizophrenia model on a 6234 CPU.

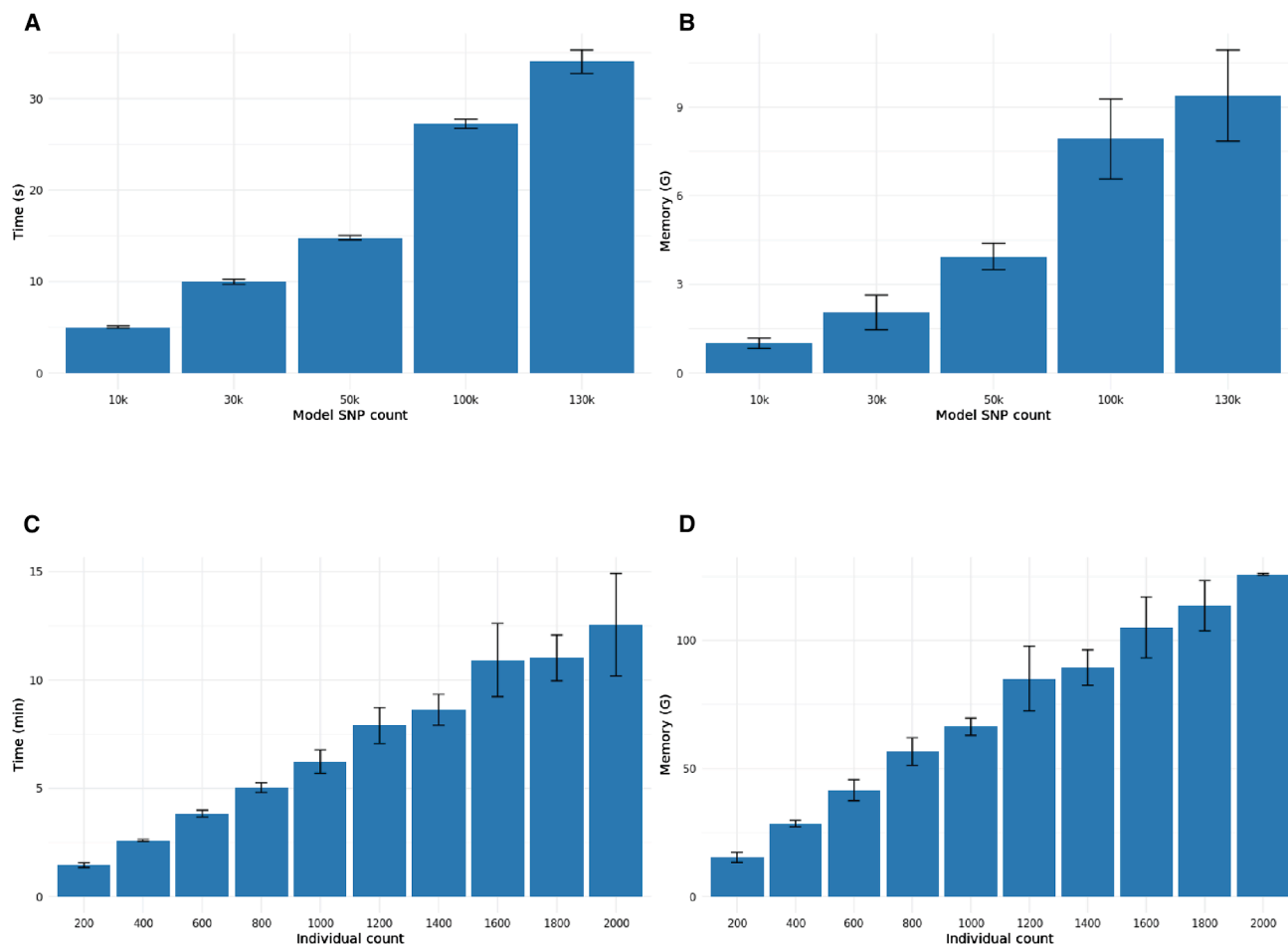


Figure 4. Time and memory scale with SNPs and cohort size

Homomorphic encryption computation time and memory usage scale approximately linearly with the number of SNPs and individuals.

(A and B) (A) Computation time and (B) memory usage as a function of the number of SNPs, generated using synthetic datasets containing a range of SNP sizes created by HAPGEN2. The runs were performed using a ring dimension 2^{13} , a sample size of 50 individuals, and phenotype 0 (as specified in Table 3; Table S1). (C and D) (C) Computation time and (D) memory usage as a function of the number of individuals, using the synthetic dataset with 130,000 SNPs and varying only the number of individuals for each run. A ring dimension of 2^{13} and phenotype 0 were used for these experiments. Error bars indicate the SD from 10 independent trials tested on a random choice from a 6234 CPU.

orders of magnitude slower than plaintext calculations (on a single CPU node Intel 6234). When we implemented the dot product calculation in Golang, it took 0.303 s; however, a more common way to calculate the PRS is with LDpred2 in R, which took only 0.073 s. When comparing the dot product calculation method within Golang, the homomorphic encryption calculation (248.3 s) took around 800 times longer. Compared to standard methods of calculating the PRS, the encrypted calculation took 3,400 times longer. This contrast highlights the computational burden introduced by encryption protocols and the possible time added by the Golang language.

Using a synthetic dataset allowed us to vary both the input sample size and the number of SNPs of the underlying PRS model to determine their effects on the performance of our method. We first assessed the influence of the number of SNPs on the time taken to calculate PRSs using FHE for 50 samples (Figure 4). We observed a direct correlation between

the increase in SNP numbers, from 10,000 to 130,000, and the computational time required, from 5 to 36 s, demonstrating an approximately linear relationship under the conditions tested (Figure 4A). Figure 4B expands on this by showing that memory requirements also rise in a similar linear fashion (from 0.8 to 9 GB) with the increase in SNP count, which is a critical consideration for the practical application of FHE in clinical settings. See also Table 3.

Next, we varied the number of individuals for which we performed the encrypted PRS calculations, using a constant genome size of 130k SNPs and predicting for artificial phenotype 0. Similar to the number of input SNPs, we found that the number of individuals included in a batch had a linear relationship with the time to completion (Figure 4C), ranging from about 1.5 min for 200 individuals to 12.5 min for 2,000 individuals. The required memory also positively correlated with the batch size, from 17 GB for 200 individuals to 125 GB for

Table 2. CKKS parameterization and security

$\log_2 N$	13	–	–	–	–	–
$\log_2(Q)$	33	27	27	27	27	27
$\log_2(P)$	34	–	–	–	–	–
$\log_2(QP)$	202	–	–	–	–	–
Default scale	2^{27}	–	–	–	–	–

Explanation of encryption parameters: we work in the cyclotomic ring $R = \mathbb{Z}_Q[X]/(X^N + 1)$. Here N is the ring dimension, $Q = \prod_i q_i$ is the ciphertext modulus (each $q_i \equiv 1 \pmod{2N}$), and P is an auxiliary prime used during key switching and relinearization. In our setup, $\log_2 N = 13$, ($N = 8,192$), $\log_2(q_i) \in \{33, 27, 27, 27, 27, 27\}$, and $\log_2(P) = 34$, so the largest modulus used at any point is $q_{\max} = QP = 2^{202}$. Security for RLWE-based HE is assessed against N and the largest modulus q that appears during the computation. With $N = 8,192$ the HE Standard's 128-bit recommendations allow roughly $\log_2(q) = 218$ for classical and $\log_2(q) = 202$ quantum.³⁷ The default scale specifies the amount to scale the plaintext during the encoding process, which impacts precision and maximum depth. We use a pretested parameter configuration specified by the Lattigo package.

2,000 individuals (Figure 4D). These data point to a predictable increase in analysis time with larger cohorts, reinforcing the scalability of our FHE protocol for PRS calculation in real-world scenarios with substantial genomic datasets.

DISCUSSION

In this work, we implemented an FHE-based method for accurate and efficient PRS calculations without compromising patient privacy. We show that the marginal error introduced by the

encryption scheme does not affect the overall error of the PRS model. We also demonstrate that different parameters affect the accuracy of results and the trade-off between accuracy and time within the FHE framework. Although FHE substantially increases the computational burden for PRS calculation compared to plaintext models, roughly three orders of magnitude slower, the cost remains within a reasonable range for practical applications. These findings suggest that our FHE-based method is viable for securely computing PRSs, while maintaining a balance between privacy, accuracy, and computational efficiency.

The bulk of our PRS calculation is performed within our function `HE_inner_product()`. While this operation is a simple calculation, the nature of large genomic data requires some reworking of encrypted data storage and the computation sequence to arrive at an accurate phenotype prediction without excessive time or memory consumption. In the [STAR Methods](#) section, we provide an intuitive explanation of how we utilize RLWE to compute the dot product.

As explained in the [STAR Methods](#), the modeler must share the encrypted model with the evaluator; if the evaluator were to share this encrypted model with the client, the client could decrypt the model without the modeler's consent, thus violating privacy requirements. The assumption that the evaluator and client never collude is reasonable if we consider the evaluator to be a healthcare entity (e.g., a hospital or healthcare insurance institution). These organizations might be associated with a research group that owns the model. We assume the evaluator would want to maintain a good relationship with the modelers and not erode trust. However, the trust between the modeler and evaluator poses a vulnerability in the framework.

Table 3. Symbols and dimensions used in the encrypted PRS pipeline

Data type	Symbol	Description	Dimension
Input data	M	number of SNPs included in PRS model	scalar
	J	number of individuals	scalar
	X_i	genomic data vector	(M)
	β	Unencrypted model vector	(M)
	security	chosen security NIST category	–
Dependent on security + M	QP	extended modulus (allows for more depth of computation with security compromise)	scalar
	N	ring dimension (determines slot capacity and chosen to meet target security for given $q_{\max}$)	scalar
	slots	maximum vector size ($N/2$)	scalar
	K	number of encryption vectors per one genome $M/(N/2)$	scalar
Calculated values	$X_{e,i}$	encrypted genomic matrix	($K, N/2$)
	β_e	encrypted PRS model matrix	($K, N/2$)
	$Y_{e,i}$	encrypted result vector	($N/2$)
	y_i	unencrypted result	scalar

N is the ring dimension, which determines the slot capacity. The modulus budget QP allows for more depth of computation and is bounded by security for a given N . Slots represents the maximum of $N/2$ values that can be encoded at once, or in other words, the maximum vector size for the given ring dimension. M is the number of SNPs included in the PRS calculation. J is the number of individuals tested. K is the number of encrypted vectors that comprise one encrypted genome $\left(1 + \frac{M-1}{\text{slots}}\right)$. X_i is the genomic vector for individual i , and $X_{e,i}$ is the encrypted genomic matrix for individual i . β is the vector containing model weights for the PRS and β_e is the model matrix with encrypted weights. $Y_{e,i}$ is the encrypted results vector, and y_i is the unencrypted result scalar that communicates an individual's risk for disease.

Approximately 4%–7% of an average healthcare system's IT budget is spent on cybersecurity. This means that from 2020 to 2025, healthcare organizations will allocate \$125B to cybersecurity.³⁶ While hospitals use a large portion of this budget to defend the Internet of Things, some of this capital is spent protecting healthcare data, which increasingly includes patient genomic data. Our protocol could significantly reduce vulnerabilities if patient genomes were stored in encrypted formats and only the decryption keys were maintained under high security. Such an encryption framework would reduce vulnerabilities because only encrypted genomic data would be transferred, rather than raw genomic data. However, privacy could still be breached if the evaluator and client colluded to reveal the underlying PRS weights.

As our pipeline assumes a single decryption authority (the client), we use one public/secret key pair shared by all parties for encryption and evaluation. This is optimal when only the genotype owner needs access to the plaintext result and the PRS model owner (researcher) trusts the evaluator. Specifically, our protocol involves three logical roles (client, modeler, evaluator), all operating under a single CKKS public key generated by the client (Figure 1). This “single-key, many-party” layout is a natural match for the encrypted inference use case we target: the client (e.g., a clinician) requests phenotype predictions on a patient's genotype without revealing raw variants to the cloud evaluator, the evaluator and model provider do not receive access to genotypic information, and the evaluator cannot decrypt the model or genotype. Thus, this method relies on the assumption that the evaluator and client will not collude to reveal the model parameters and assumes trust between the evaluator and modeler. However, many genomic workflows involve multiple mutually distrustful custodians, such as a consortium of hospitals storing data for a subset of patients or two companies wishing to jointly train a PRS without exposing proprietary cohorts. In such cases, a multiparty (collective-key) CKKS scheme would remove the need to trust any single organization with the full secret key. In this context, the secret key is additionally shared across the parties; no single party can decrypt, but a quorum can run a threshold-decryption (or key-switch) protocol. This mitigates the “evaluator and modeler trust” assumption at the cost of increased interaction and computation. True multiparty CKKS (collective-key or multiparty computation FHE) requires interactive key generation and online key-switch refreshes, introducing $>3\times$ communication overhead. This would add runtime and complexity, which is unnecessary in our current context where only the client requires decryption capability. However, a multiparty-FHE PRS pipeline is a possible next step and could be valuable in different contexts where a federated learning process is desired, such as when there are multiple modelers or when the model provider and evaluator do not trust each other.

In this work, we demonstrated the feasibility and practicality of generating PRS model predictions without revealing a genome. We showed how memory requirements increase with higher accuracy specifications and how runtime scales roughly linearly with the number of SNPs included in the

model. Last, we showed that the error introduced by homomorphic encryption is negligible compared to the inherent error in the model itself.

Considering recent advances in FHE, we view PRSs as an exciting new avenue for applying this technology. The strategic use of genomic information has the potential to significantly contribute to personalized healthcare to enhance health outcomes and elevate care standards. The necessity for patients to undergo genomic sequencing is paramount for realizing this technology's benefits. Concurrently, the escalation of genetic data mandates stringent security and privacy measures. The challenges of applying analytical models to decipher relevant information from the large-scale data required for genome encoding are non-trivial. Our research offers a solution that addresses these multifaceted issues. While our focus is on PRSs, the principles of FHE can potentially be applied to a spectrum of genomic computations. The generalizability of FHE to other areas in clinical genomics, such as variant annotation, genotype imputation, and more complex predictive modeling, warrants further investigation. This work takes an initial step toward exploring such applications.

Limitations of the study

Our protocol assumes a single decryption authority and non-collusion between client and evaluator; it does not use multiparty/collective-key CKKS. Runtime and memory overheads are larger than plaintext, and our PRS implementation does not explicitly model LD to the extent of LDpred2. These factors may limit immediate deployment in settings with strict latency or federated trust requirements.

RESOURCE AVAILABILITY

Lead contact

Further information and requests for resources and reagents should be directed to and will be fulfilled by the lead contact, Mark Gerstein (mark@gersteinlab.org).

Materials availability

This study did not generate new unique reagents or physical materials.

Data and code availability

- Synthetic example data used for demonstration are available with the HEPRS code repository. PsychENCODE genotypes are accessible to approved researchers at <http://resource.psychencode.org/>. Schizophrenia GWAS summary statistics are available from the Psychiatric Genomics Consortium (<https://pgc.unc.edu/for-researchers/download-results>). Accession IDs/links are provided in the [key resources table](#).
- HEPRS (Go implementation; CKKS/Lattigo) is publicly available at <https://github.com/gersteinlab/HEPRS>, along with a small example dataset that includes synthetic genotypes and phenotypes generated using HAPGEN2 for tutorial purposes. An archival version is available at Zenodo: <https://doi.org/10.5281/zenodo.17486063>.
- Any additional information required to re-analyze the data reported in this paper is available from the [lead contact](#) upon request.

ACKNOWLEDGMENTS

This work was supported by the National Institutes of Health (R01HG010749) and the Albert L Williams Professorship funds.

AUTHOR CONTRIBUTIONS

Conceptualization, M.G., E.K., J.L., and C.K.; methodology, M.G., E.K., J.L., C.K., and M.J.; software, J.L., E.K., M.J., and I.Y.; validation, J.L., M.J., and I.Y.; formal analysis, M.G., J.L., E.K., and M.J.; investigation, M.G., E.K., J.L., M.J.; data curation, J.L. and M.J.; writing – original draft, M.G., E.K., J.L., and M.J.; writing – review & editing, M.G., E.K., J.L., M.J., and I.Y.; supervision, M.G.; funding acquisition, M.G.

DECLARATION OF INTERESTS

The authors declare that they have no competing interests.

STAR★METHODS

Detailed methods are provided in the online version of this paper and include the following:

- **KEY RESOURCES TABLE**
- **METHOD DETAILS**
 - Technical overview of key aspects of FHE for PRS
 - Key generation
 - Synthetic data generation and ridge regression
 - Schizophrenia PRS generation
 - Ring dimension affects accuracy and runtime for PRS calculation
- **QUANTIFICATION AND STATISTICAL ANALYSIS**

SUPPLEMENTAL INFORMATION

Supplemental information can be found online at <https://doi.org/10.1016/j.crmeth.2025.101271>.

Received: March 12, 2025

Revised: August 13, 2025

Accepted: December 3, 2025

Published: January 8, 2026

REFERENCES

1. McCombie, W.R., McPherson, J.D., and Mardis, E.R. (2018). Next-Generation Sequencing Technologies. *Cold Spring Harb. Perspect. Med.* 9, a036798.
2. Uffelmann, E., Huang, Q.Q., Munung, N.S., de Vries, J., Okada, Y., Martin, A.R., Martin, H.C., Lappalainen, T., and Posthuma, D. (2021). Genome-wide association studies. *Nat. Rev. Methods Primers* 1, 59.
3. Manolio, T.A. (2013). Bringing genome-wide association findings into clinical use. *Nat. Rev. Genet.* 14, 549–558.
4. Finan, C., Gaulton, A., Kruger, F.A., Lumbers, R.T., Shah, T., Engmann, J., Galver, L., Kelley, R., Karlsson, A., Santos, R., et al. (2017). The druggable genome and support for target identification and validation in drug development. *Sci. Transl. Med.* 9, eaag1166.
5. Lewis, C.M., and Vassos, E. (2020). Polygenic risk scores: from research tools to clinical instruments. *Genome Med.* 12, 44.
6. Sugrue, L.P., and Desikan, R.S. (2019). What Are Polygenic Scores and Why Are They Important? *JAMA* 321, 1820–1821.
7. Marston, N.A., Pirruccello, J.P., Melloni, G.E.M., Koyama, S., Kamanu, F.K., Weng, L.C., Roselli, C., Kamatani, Y., Komuro, I., Aragam, K.G., et al. (2023). Predictive Utility of a Coronary Artery Disease Polygenic Risk Score in Primary Prevention. *JAMA Cardiol.* 8, 130–137.
8. Mavaddat, N., Michailidou, K., Dennis, J., Lush, M., Fachal, L., Lee, A., Tyrer, J.P., Chen, T.H., Wang, Q., Bolla, M.K., et al. (2019). Polygenic Risk Scores for Prediction of Breast Cancer and Breast Cancer Subtypes. *Am. J. Hum. Genet.* 104, 21–34.
9. Yengo, L., Vedantam, S., Marouli, E., Sidorenko, J., Bartell, E., Sakaue, S., Graff, M., Eliassen, A.U., Jiang, Y., Raghavan, S., et al. (2022). A saturated map of common genetic variants associated with human height. *Nature* 610, 704–712.
10. Zhang, H., Ahearn, T.U., Lecarpentier, J., Barnes, D., Beesley, J., Qi, G., Jiang, X., O'Mara, T.A., Zhao, N., Bolla, M.K., et al. (2020). Genome-wide association study identifies 32 novel breast cancer susceptibility loci from overall and subtype-specific analyses. *Nat. Genet.* 52, 572–581.
11. Shi, X., and Wu, X. (2017). An overview of human genetic privacy. *Ann. N. Y. Acad. Sci.* 1387, 61–72.
12. Bycroft, C., Freeman, C., Petkova, D., Band, G., Elliott, L.T., Sharp, K., Motyer, A., Vukcevic, D., Delaneau, O., O'Connell, J., et al. (2018). The UK Biobank resource with deep phenotyping and genomic data. *Nature* 562, 203–209.
13. Freund, K. (2022). IBM Announces Two Innovations To Advance Quantum Computing. November 10, 2022. <https://www.forbes.com/sites/karlfreund/2022/11/10/ibm-announces-two-innovations-to-advance-quantum-computing>.
14. Wang, Y., Si, C., and Wu, X. (2015). Regression model fitting under differential privacy and model inversion attack. *Proceedings of the 24th International Conference on Artificial Intelligence. IJCAI'15 (AAAI Press)*, pp. 1003–1009.
15. Kara, M., Laouid, A., Yagoub, M.A., Euler, R., Medileh, S., Hammoudeh, M., Eleyan, A., and Bounceur, A. (2022). A fully homomorphic encryption based on magic number fragmentation and El-Gamal encryption: Smart healthcare use case. *Expert Syst.* 39, e12767.
16. Raisaro, J.L., Choi, G., Pradervand, S., Colsenet, R., Jacquemont, N., Rosat, N., Mooser, V., and Hubaux, J.P. (2018). Protecting Privacy and Security of Genomic Data in i2b2 with Homomorphic Encryption and Differential Privacy. *IEEE/ACM Trans. Comput. Biol. Bioinform.* 15, 1413–1426.
17. Naveed, M., Ayday, E., Clayton, E.W., Fellay, J., Gunter, C.A., Hubaux, J.P., Malin, B.A., and Wang, X. (2015). Privacy in the Genomic Era. *ACM Comput. Surv.* 48, 6.
18. Bonomi, L., Huang, Y., and Ohno-Machado, L. (2020). Privacy challenges and research opportunities for genomic data sharing. *Nat. Genet.* 52, 646–654.
19. Kim, M., and Lauter, K. (2015). Private Genome Analysis through Homomorphic Encryption. *Proceedings of the 14th International Conference on Financial Cryptography and Data Security (Springer)*, pp. 239–256. https://doi.org/10.1007/978-3-662-48185-7_14.
20. McLaren, P.J., Raisaro, J.L., and Aouri, M. (2016). Privacy-Preserving Genomic Analysis for Personalized HIV Treatment. *Proceedings of the 7th International Workshop on Data Privacy in Healthcare (ACM)*, pp. 1–10. <https://doi.org/10.1145/2983655.2983659>.
21. Blatt, M., Gusev, A., Polyakov, Y., and Goldwasser, S. (2020). Secure large-scale genome-wide association studies using homomorphic encryption. *Proc. Natl. Acad. Sci. USA* 117, 11608–11613.
22. Kaushik, A. (2023). Investigating Resilience of Levelled Fully Homomorphic Encryption System Against Data Scientific Attacks. Master's thesis. <https://akaushik.edufor.me/resources/MasterThesis.pdf>.
23. Nyland, S.P. (2023). Implementation and Evaluation of a Private Set Intersection Protocol Built With Fully Homomorphic Encryption. Master's thesis. <https://repository.tugraz.at/publications/marc21/0x1n4-ft584/files/77411.pdf>.
24. Gursoy, G., Ozkaya, K., Demirci, M., and Yilmaz, E. (2021). Privacy-preserving Genotype Imputation with Fully Homomorphic Encryption. *BMC Genom.* 22, 101.
25. (2023). Lattigo v5. <https://github.com/tuneinsight/lattigo>.
26. Human Genome Privacy Challenge 2022.2022 Accessed: September 2022. 2022. url: <http://www.humangenomeprivacy.org/2022/>
27. Islam, S., Demirci, H., and Lenzini, G. (2022). Privacy-preserving Polygenic Risk Scoring using Homomorphic Encryption. *Proceedings of the 18th Scandinavian Conference on Health Informatics, 187*, pp. 124–127. <https://doi.org/10.3384/ecp187021>.
28. Trubetskoy, V., Pardiñas, A.F., Qi, T., Panagiotaropoulou, G., Awasthi, S., Bigdeli, T.B., Bryois, J., Chen, C.Y., Dennison, C.A., Hall, L.S., et al. (2022).

Mapping genomic loci implicates genes and synaptic biology in schizophrenia. *Nature* 604, 502–508.

29. Kim, A., Papadimitriou, A., and Polyakov, Y. (2021). Approximate Homomorphic Encryption with Reduced Approximation Error (New Jersey Institute of Technology, Samsung Advanced Institute of Technology, Duality Technologies). <https://eprint.iacr.org/2020/1118.pdf>.
30. Su, Z., Marchini, J., and Donnelly, P. (2011). HAPGEN2: simulation of multiple disease SNPs. *Bioinformatics* 27, 2304–2305.
31. Wang, D., Liu, S., Warrell, J., Won, H., Shi, X., Navarro, F.C.P., Clarke, D., Gu, M., Emani, P., Yang, Y.T., et al. (2018). Comprehensive functional genomic resource and integrative model for the human brain. *Science* 362, eaat8464.
32. Craig, D.W., Goor, R.M., Wang, Z., Paschall, J., Ostell, J., Feolo, M., Sherry, S.T., and Manolio, T.A. (2011). Assessing and managing risk when sharing aggregate genetic variant data. *Nat. Rev. Genet.* 12, 730–736.
33. Chang, C.C., Chow, C.C., Tellier, L.C., Vattikuti, S., Purcell, S.M., and Lee, J.J. (2015). Second-generation PLINK: rising to the challenge of larger and richer datasets. *GigaScience* 4, 7.
34. Choi, S.W., Mak, T.S.H., and O'Reilly, P.F. (2020). Tutorial: a guide to performing polygenic risk score analyses. *Nat. Protoc.* 15, 2759–2772.
35. Prive, F., Arbel, J., and Vilhjálmsson, B.J. (2021). LDpred2: better, faster, stronger. *Bioinformatics* 36, 5424–5431.
36. Morgan, S. (2020). Healthcare industry to spend \$125 billion on cybersecurity from 2020 to 2025. <https://cybersecurityventures.com/healthcare-industry-to-spend-125-billion-on-cybersecurity-from-2020-to-2025>.
37. Albrecht, M., Chase, M., and Chen, H. (2018). Homomorphic Encryption Standard v1.1. Tech. rep. HomomorphicEncryption.org. <https://homomorphicencryption.org/wp-content/uploads/2018/11/HomomorphicEncryptionStandardv1.1.pdf>.
38. Albrecht, M., Chase, M., and Chen, H. (2019). Homomorphic Encryption Standard. Cryptology ePrint Archive, Report 2019/939. <https://eprint.iacr.org/2019/939>.
39. CIC/IACR (2022). Security Guidelines for Homomorphic Encryption. <https://cic.iacr.org/p/1/4/26.2022>.
40. Li, N., and Stephens, M. (2003). Modelling Linkage Disequilibrium, And Identifying Recombination Hotspots Using SNP Data. *Genetics* 165, 2213–2233.
41. Shi, H., Burch, K.S., Johnson, R., Freund, M.K., Kichaev, G., Mancuso, N., Manuel, A.M., Dong, N., and Pasaniuc, B. (2020). Localizing Components of Shared Transethnic Genetic Architecture of Complex Traits from GWAS Summary Data. *Am. J. Hum. Genet.* 106, 805–817.
42. Stovner, E.B., and Cole, B.S. (2019). snpflip. <https://github.com/biocore-ntnu/snpflip>.

STAR★METHODS

KEY RESOURCES TABLE

REAGENT or RESOURCE	SOURCE	IDENTIFIER
Deposited data		
PsychENCODE genotypes (controlled access) used for schizophrenia cohort	PsychENCODE Consortium	PsychENCODE: http://resource.psychencode.org/ ; NIMH Data Archive: 5032; DOI: https://doi.org/10.15154/1g4m-dy13
Schizophrenia GWAS summary statistics	Psychiatric Genomics Consortium (PGC); Trubetskoy ²⁸	PGC: PGC3SCZ; URL: https://pgc.unc.edu/for-researchers/download-results
Processed schizophrenia PRS weights used in this study	This paper	Not publicly available.
Synthetic example data (tutorial genotypes/phenotypes)	This paper (deposited with code)	GitHub: https://github.com/gersteinlab/HEPRS . Zenodo: https://doi.org/10.5281/zenodo.17486063
HapMap3 reference SNP set/1000G genetic map used by LDpred2	bigsnpr/LDpred2 resources	bigsnpr/LDpred2: https://privefl.github.io/bigsnpr/articles/LDpred2.html
Software and algorithms		
HEPRS (Go implementation; CKKS/Lattigo)	This paper; Gerstein Lab	GitHub: https://github.com/gersteinlab/HEPRS ; Zenodo: https://doi.org/10.5281/zenodo.17486063
Lattigo (CKKS) v3.0.5	Tune Insight/LDSEC	GitHub: https://github.com/ldsec/lattigo/releases ; Version: v3.0.5
Go toolchain v1.20.4	The Go Project	Go: 1.20.4; URL: https://go.dev/dl/
PLINK 2	Chang et al. ³³ ; COG Genomics	PLINK 2: 2.3.11; URL: https://www.cog-genomics.org/plink/2.0/
bigsnpr/LDpred2	Privé et al. ³⁵	bigsnpr: https://privefl.github.io/bigsnpr/
snpflip (strand harmonization)	biocore-ntnu	GitHub: https://github.com/biocore-ntnu/snpflip ; PyPI: https://pypi.org/project/snpflip/ GitHub: https://github.com/biocore-ntnu/snpflip ; PyPI: https://pypi.org/project/snpflip/
HAPGEN2 (genotype simulation)	Oxford/WTCHG (Marchini group)	Oxford MathGen: https://mathgen.stats.ox.ac.uk/genetics_software/hapgen/hapgen2.html ; PMID: PMC3150040
scikit-learn (AUROC/metrics)	scikit-learn developers	scikit-learn: 1.1.3; URL: https://scikit-learn.org

METHOD DETAILS

Technical overview of key aspects of FHE for PRS

We use the CKKS encryption scheme, implemented in the Lattigo library,²⁵ as the FHE method in our encrypted PRS calculation framework. The CKKS scheme gains NP-hard security by leveraging the RLWE problem. Homomorphic evaluation allows the scheme to perform a specific set of operations on ciphertexts, such as addition and multiplication, and produce a new ciphertext as a result. The CKKS scheme is classified as lattice-based cryptography and is quantum-secure.²⁹ Our parameter set meets 2^{128} classical and quantum gate complexity, satisfying NIST category one security.³⁷ Although lattice problems do admit a quadratic speed-up in quantum models, this does not translate into a large reduction in security levels. As shown in the Homomorphic Encryption Standard³⁸ and the CIC/IACR guidelines,³⁹ the difference between classical and quantum estimates is only a few bits. For example, parameters yielding ~ 128 bits of classical security typically yield 124–127 bits quantum, not a large drop.

Table 3 outlines each important parameter and its meaning in the context of the encrypted PRS pipeline. We first encode the model β and the genotype vector of each individual X_i for J individuals as CKKS plaintexts; CKKS packs a real vector X_i into a polynomial of degree N with coefficients in the ring $\frac{\mathbb{Z}_q[X]}{X^{N+1}}$ (this is also stored in vector format $X_{e,i}$). These plaintexts are then encrypted to produce the ciphertext matrices used in the subsequent homomorphic computations (see Methods 3.6 for details).

Figure 2 outlines how we encode the genomic vector into the encrypted domain to prepare for FHE evaluation. During encoding, we break each individual's genome vector X_i with dimension $1 \times M$ into a matrix of dimension $K \times N/2$. This becomes matrix $X_{e,i}$ with dimension $K \times N/2$. After performing this process for each individual, we generate a three-dimensional tensor of dimension $J \times K \times N/2$. We repeat this process for vector β to obtain matrix β_e with dimension $K \times N/2$. We ultimately compute the calculation $X_i \times \beta = y_i$, where y_i is the scalar output from the PRS model. We outline our algorithm in Algorithm 1 with a greater explanation of the individual steps in Methods Section 3.6.

In Table 2, we highlight the important parameters that customize the CKKS encryption scheme for optimal results for a given task. The parameter N , in the context of CKKS (and other homomorphic encryption schemes), represents the degree of the polynomial ring over which the encryption operates. CKKS encodes vectors into polynomials of a degree defined by N (we provide further explanations in the Methods section). The ring dimension N is chosen to satisfy a greater than 128-bit classical security for a given $\log_2(Q)$. The value of N is typically chosen to be a power of 2 for efficient implementation, mainly because fast Fourier transform algorithms, which are used for polynomial multiplication, are most efficient when operating on sequence lengths that have a power of 2. The values of Q and P determine the lowest possible value of N to maintain security (as larger N increases computational cost). The genomic vectors are converted into a matrix when encoded into the encrypted domain defined by a cyclotomic polynomial, whose domain is defined by Q and N . Q determines the precision of encrypted genetic data, ensuring accurate PRS computation without loss of data integrity. P is critical during the relinearization step post-multiplication to safeguard the computation's accuracy. We describe each parameter in Table 2.

Key generation

The key generation step is implemented using the `Encrypt_input` function in the Lattigo library. In this function, we employ the CKKS parameters to create a new key generator through the "NewKeyGenerator" method provided in the CKKS library within Lattigo. The "GenKeyPair" method generates a public-private key pair (pk and sk), and the "GenRelinearizationKey" method creates the relinearization key that enables efficient computation on encrypted data. This method requires the secret key and `max_level` parameters, the latter of which we set to 2 (to support addition and subtraction calculations). Finally, the "GenRotationKeysForInnerSum" method generates rotation keys, which allow operations to include rotations of encrypted data (for instance, during the inner product calculation). The function returns the generated keys and parameters for further use in the encryption and evaluation steps of the homomorphic encryption scheme.

Synthetic data generation and ridge regression

Hapgen2³⁰ is a simulation method that resamples known haplotypes quickly and efficiently to produce samples with linkage disequilibrium (LD) patterns that mimic those in real data. Hapgen2 is based on the Li and Stephens model⁴⁰ of LD. The method takes in a reference panel of haplotypes as input and generates genotype encodings based on the haplotypes. We used Hapgen2 to simulate genotype encodings for 40,000 individuals across a varying number of SNP sizes, (30k, 50k, 100k, and 130k), based on the 1000 Genomes project reference panel. We later split these individuals into training and testing groups for ridge regression of sizes 30,000 and 2,000, respectively, for each SNP size.

We considered a high-dimensional regression framework for polygenic modeling and prediction

$$Y = X\beta + \epsilon \quad (\text{Equation 1})$$

where Y is an $N \times 1$ vector of phenotype values with $Y_i \in \mathbb{R}$ and N is the number of individuals in the dataset. The parameter $X^{N \times M}$ is the genotype encoding matrix with $X_{im} \in \{0, 1, 2\}$ and M is the number of SNPs in the genotype encoding. The corresponding effect size of each SNP m is β_m . The parameter ϵ is a vector of residual effects for each individual. We are interested in estimating β . We assign a Gaussian prior on each β_m .

$$\beta_i \sim N\left(0, \sqrt{h^2/(Mp)}\right) \quad (\text{Equation 2})$$

where p is the percentage of SNPs with non-null effect size, and h^2 is the total SNP-heritability. Hence, we assume that each SNP with a non-zero effect size explains an equal portion of the total SNP-heritability. To model the impact of SNPs with zero effect size, we select $M(1-p)$ random SNP effect size locations in the β vector and set the value at those indexes equal to zero. Each of the three artificial phenotypes were generated under the assumption that approximately 90% of the SNPs had non-zero effect size, or $p = 0.9$, a level of sparsity generally consistent with realistic human data.⁴¹ The three levels of SNP heritability used were $h^2 = 0.3, 0.6, 0.9$.

We used multiple SNP sizes, (10k, 30k, 50k, 100k, and 130k), for our simulation data to show that the FHE scheme would apply well to real-world settings where the SNP size varies depending on which traits are being studied. Both values of p and h^2 were fixed across the training and testing populations to maintain consistency across the two data populations. The population count of the training dataset was $N = 30,000$, and for the testing data the count was $N = 2,000$.

Recall that the estimator of Ridge Regression $\hat{\beta}$ is equal to the mean of the posterior distribution under a Gaussian prior. Hence, the choice of a Gaussian prior on β in this PRS setting naturally leads us to use ridge regression. Moreover, an extra column of 1s was added for the intercept term. We note that ridge regression is one of many models researchers use for calculating PRS. More advanced models, such as Bayesian hierarchical models using Gibbs sampling or variational inference, may be more accurate

because they have relaxed assumptions on the parameters of the training data. Although we sacrifice computational accuracy in using ridge regression, we gain faster time complexity. In real-world settings, this trade-off is not as important since the model is built once off of training data and not repeatedly calculated. It is important to note that the sacrifice on accuracy is made in the model selection and not in the FHE scheme. Hence, the FHE scheme's performance should be judged purely on the accuracy of the decrypted prediction values compared to the ciphertext prediction values.

Schizophrenia PRS generation

We calculated schizophrenia PRSs for 1,146 individuals from the PsychENCODE Consortium study, including 493 individuals diagnosed with schizophrenia and 653 controls.³¹ Imputed genotypes and metadata for these individuals were downloaded from the PsychENCODE Consortium data portal,³¹ with more than 4.3M imputed SNPs available for each individual. Schizophrenia GWAS summary statistics were selected from a recent Psychiatric Genomics Consortium (PGC) study of more than 320,000 individuals and were downloaded from the PGC portal.²⁸ Using these inputs, we implemented a standardized data processing pipeline for calculating PRSs based on best practice guidelines.³⁴ We first performed quality control filtering on the GWAS summary statistics by removing SNPs with INFO scores less than 0.8, as well as ambiguous and duplicate SNPs. Next, after lifting over coordinates and fixing alleles of the PsychENCODE genotypes to the hg38 reference genome, we performed strand-flipping using the snpflip software package.⁴² We removed SNPs with minor allele frequency < 0.05 or Hardy-Weinberg equilibrium $p < 1 \times 10^{-6}$ (resulting in more than 2.6M SNPs per sample) and removed individuals with > 3 SD genotype heterozygosity rate (F coefficient, calculated using SNPs in 200 kb windows with $LD\ r^2 > 0.25$). An additional strand-flipping step was performed using snpflip to match alleles between the sample genotypes and summary statistic SNPs. Finally, six genotype principal components (PCs) were calculated for each sample using the `-pca` command in PLINK2.³³

We used the LDpred2 function within the bigsnpr software package to calculate PRSs for each individual.³⁵ After filtering for SNPs present in the HapMap3 dataset, we calculated LD scores for 110,258 matching SNPs between the PsychENCODE genotypes and summary statistics using the `snp_cor` function and centimorgan map units from the 1000 Genomes project. We then regressed the LD scores against the log-scaled summary beta values using the `snp_ldsc` function and calculated a baseline heritability estimate based on the output (h^2_{est} , here equal to 0.1270). Using these modified summary statistics, we calculated PRSs for each individual using the infinitesimal and grid models of LDpred2. For the grid model, we assessed combinations of (a) 17 input SNP p -value filters ranging from 1.0×10^{-4} to 1.0, (b) three potential values of h^2 (0.7, 1.0, and $1.4 \times h^2_{est}$), and (c) sparse or non-sparse grid models, for a total of 102 PRS outputs. (We note that the LDpred2-auto model did not converge for our datasets and thus was not used in our analysis.) To assess the performance of each PRS model, we generated logistic regression models for PRSs toward the binary schizophrenia diagnosis of each individual, with sex, age, and six genotype PCs included as covariates. We then calculated Nagelkerke $pseudo-R^2$ values for each regression model, and compared them with those for a null logistic regression model consisting solely of the sex, age, and genotype PC covariates.

Overall, we found that both the infinitesimal model ($pseudo-R^2 = 0.2008$) and best-performing grid model ($pseudo-R^2 = 0.2229$; at $p = 0.32$, $h^2 = 0.7 \times h^2_{est}$, and sparse grid model) performed substantially better than the null model ($pseudo-R^2 = 0.0809$). We selected PRS outputs derived from the best-performing grid model for comparison with encrypted PRS calculations. The correlations and AUROC calculations performed in this analysis were completed in Python using the sklearn library.

In contrast to the LDpred2 model, our PRS calculation approach used for the homomorphic encryption protocol performs SNP filtering as a preprocessing measure. In this way, we ensure that the input to our model consists of already filtered SNPs. Additionally, our model does not explicitly manage LD in the same detailed manner. Instead, it operates similarly to a straightforward PRS model, which assumes that the selected SNPs are representative of all SNPs that contribute to a phenotype and have been pre-processed to mitigate LD concerns.

Ring dimension affects accuracy and runtime for PRS calculation

We show that increasing the ring dimension N parameter of the CKKS encryption scheme will decrease the errors introduced in the encrypted calculations S2. However, increasing the ring dimension will also increase the model runtime S2.

From the results obtained with the 110K-SNP schizophrenia PRS model, we observed that the time required for the encrypted model to generate PRS predictions increased with the ring dimension (Figure S1A). Evaluating PRSs for 1,146 individuals using the 110K-SNP model in plaintext (infinitesimal mode of LDpred2 for only the evaluation part) typically takes 0.07 s on a single CPU (Intel 6234) (Table S2), while for FHE, even using the smallest ring dimension 2^{13} takes about 6 min (Figure S1A), and the time further increases to 20 min for the ring dimension 2^{16} . When breaking down the time cost into different stages of the encryption calculation, we find that most of the time is spent after the encryption step and during the model operation phase, corresponding to the actual computation of the encrypted PRS (Figure S1B).

Figure S1C presents the memory allocation for running the program across different ring dimensions. Contrary to what we expected, the memory usage remains largely invariant through repeated experiments, ranging from 60 GB to 80 GB, which only introduces a moderate increase in memory usage compared to the plaintext model, which is 50 GB. Thus, our FHE method for PRS calculation allows end users to account for the trade-off between protocol time and error mitigation according to their needs by customizing the ring dimension parameter.

For both synthetic and real datasets, we observe a decrease in MSE as we increase the ring dimension N of the encryption scheme from 2^{13} to 2^{16} , which suggests a reduction in computational error with higher ring dimension. For example, the MSE reduces to 1.43×10^{-6} for the real dataset with a ring dimension of 2^{16} (Table S2). This suggests that a higher ring dimension can mitigate computational errors introduced by the encryption. Furthermore, we show in Figure 3A that all calculation methods maintain a nearly constant AUROC. In Figure 3B, we show a Pearson correlation with $r > 0.999$ between the encrypted and non-encrypted schizophrenia PRS regardless of the ring dimension.

QUANTIFICATION AND STATISTICAL ANALYSIS

This study reports a statistical genetics method detailed in the sections above. Key evaluation metrics included Pearson correlation between encrypted and plaintext PRS values, mean squared error (MSE), Nagelkerke pseudo- R^2 from logistic regression models (with PRS, sex, age, and genotype PCs as covariates), and AUROC for case/control discrimination. Encrypted computations were implemented in Go with Lattigo (CKKS). Further details are provided in the figure legends (e.g., Figures 3 and S1, S2) and Table S2.